

CacheLayer: Semantic Caching for Large Language Model APIs via Embedding Similarity

Syed Zain Rizvi
University of Ottawa
srizv034@uottawa.ca

Abstract

LLM APIs charge per token, and users frequently submit semantically equivalent queries with different surface forms, resulting in redundant API calls. Exact match caching fails to address this because it compares strings rather than meaning. We present CacheLayer, a network-level semantic caching proxy that intercepts LLM API requests, embeds incoming queries using the all-MiniLM-L6-v2 sentence transformer, and returns cached responses for semantically similar queries without contacting the upstream provider. CacheLayer implements the OpenAI chat completions specification and supports both OpenAI and Anthropic providers, requiring no application code changes to adopt. It additionally supports multi-tenant deployments with per-user API key authentication and isolated cache stores. We evaluate six cosine similarity thresholds across 15 paraphrased query pairs spanning enterprise software, finance, and software engineering domains. A threshold of 0.80 achieves a 93.3% cache hit rate while maintaining semantic precision, with hit rate declining sharply above this value. Cache hits return in 8.3ms versus approximately 1,800ms for live API calls, a 216 $\times$ latency reduction. At enterprise scale, the system projects annual savings exceeding \$20,000 for high-traffic deployments.

1 Introduction

Running large language models in production is expensive. OpenAI charges per token, Anthropic charges per token, and every other major provider does the same. For applications with real user traffic this adds up quickly. An enterprise software company deploying an internal knowledge assistant to 500 employees, each submitting an average of 20 queries per day at 500 tokens per response, spends roughly \$50 daily on GPT-4o output alone, before accounting for input tokens or system prompts.

The straightforward fix is caching. Store the response the first time a query is answered, return it instantly the next time the same query comes in. The problem is that users do not ask questions the same way twice. A solutions engineer asks “what is the implementation timeline for Salesforce Service Cloud.” A project manager on the same team asks “how long does a typical Salesforce Service Cloud deployment take.” A consultant asks “what is the average delivery timeline for a Service Cloud project.” These are three phrasings of the same question with the same correct answer, but they share no common string. Exact match caching treats them as three separate queries and makes three separate API calls. This is not a rare edge case. In enterprise software deployments, consulting firms, and customer-facing support tools, the same questions surface repeatedly across different users and teams. A pre-sales team using an AI assistant will see the same objections and feature inquiries rephrased dozens of times per week. The cost scales with the number of users, not the number of distinct questions, which is the fundamental inefficiency that motivates this work.

Semantic caching addresses this by comparing what queries mean rather than what they say. By converting queries to dense vector representations using a sentence embedding model and measuring cosine similarity between them, a caching layer can identify when two differently worded queries are asking the same thing. If a sufficiently similar query has already been answered, the cached response is returned without touching the API.

The core challenge is threshold selection. Set the similarity cutoff too low and the cache starts returning wrong answers for queries that are similar but not equivalent. Set it too high and the cache misses most paraphrases and stops being useful. Finding the right threshold is not obvious and depends on the embedding model, the query domain, and the acceptable rate of false positives.

This paper presents CacheLayer, a semantic caching proxy for LLM APIs. CacheLayer implements the OpenAI chat completions specification, meaning any application already using the OpenAI Python SDK can switch to it by changing a single URL. Incoming queries are embedded using all-MiniLM-L6-v2 [1], compared against cached entries via cosine similarity, and either served from cache or forwarded to the real API depending on whether a match is found above the configured threshold.

We make four contributions. First, we describe the system design of CacheLayer including its multi-tenant architecture with per-user API key authentication and isolated cache stores. Second, we formalize threshold selection as a precision-recall tradeoff and explain why it is the central design parameter of any semantic caching system. Third, we present an empirical analysis of six similarity thresholds across 15 paraphrased query pairs spanning three domains, identifying a cosine similarity threshold of 0.80 as the optimal operating point for general-purpose query caching with the all-MiniLM-L6-v2 model, achieving a 93.3% cache hit rate while maintaining semantic precision. Fourth, we report latency benchmarks showing that cache hits return in 8.3ms, a 216 $\times$ reduction relative to live API calls.

2 Background

2.1 Sentence Embeddings

A sentence embedding is a fixed-length vector representation of a piece of text that encodes its semantic meaning. Two sentences with similar meanings produce vectors that are geometrically close to each other, regardless of how differently they are worded [2]. This property is what makes sentence embeddings useful for semantic search, duplicate detection, and cache lookup.

CacheLayer uses all-MiniLM-L6-v2 [1], a 22 million parameter model from the Sentence Transformers library trained to produce 384-dimensional embeddings. We chose this model for three reasons. First, it is fast enough for real-time inference without a GPU. Second, it performs well on semantic textual similarity benchmarks relative to its size. Third, its small footprint means it can run as part of the proxy server without significant memory overhead.

All embeddings are L2-normalized before storage and comparison, meaning every vector is scaled to have a magnitude of exactly 1. This normalization step is important because it allows cosine similarity to be computed as a simple dot product, which is faster than the full cosine formula and produces identical results on normalized vectors.

2.2 Cosine Similarity

Cosine similarity measures the angle between two vectors rather than the distance between them. For two vectors $\mathbf{a}$ and $\mathbf{b}$ it is defined as:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| \times |\mathbf{b}|} \quad (1)$$

The result ranges from -1 to 1 . A score of 1 means the vectors point in exactly the same direction, indicating identical or near-identical meaning. A score of 0 indicates orthogonality and no meaningful relationship. Because CacheLayer normalizes all embeddings to unit length, the denominator is always 1 and the computation reduces to:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{a} \cdot \mathbf{b} \quad (2)$$

In our experiments, paraphrased queries consistently scored between 0.75 and 0.85 on this measure using all-MiniLM-L6-v2. Unrelated queries scored below 0.3 . This separation is what makes threshold-based caching viable.

2.3 Related Work

GPTCache [3] is the most directly related prior work. It is a Python library that adds caching to LLM API calls and supports both exact match and embedding-based similarity search. However, GPTCache is implemented as an application-level library that must be integrated into each codebase individually. CacheLayer takes a different approach, operating as a network-level proxy that requires no application code changes. Additionally, GPTCache does not publish a systematic analysis of threshold selection and its effect on hit rate across query domains, which is the primary empirical contribution of this paper.

LiteLLM [4] and Portkey are LLM proxy services that support basic caching, but their caching implementations use exact string matching rather than semantic similarity. They are primarily designed for routing, load balancing, and provider abstraction rather than cost reduction through semantic deduplication.

The underlying technique of using dense retrieval for semantic matching is well established in the information retrieval literature [2, 5]. Our contribution is not the embedding or similarity computation itself, but the application of these techniques to LLM API caching, the proxy architecture that makes adoption frictionless, the multi-tenant infrastructure supporting enterprise deployments, and the empirical characterization of the threshold-hit rate tradeoff.

3 System Design

3.1 Architecture

CacheLayer operates as a network-level proxy between a client application and an LLM API provider. The client sends requests to CacheLayer using the standard OpenAI chat completions format. CacheLayer intercepts each request, performs a semantic cache lookup, and either returns a cached response or forwards the request to the real API. From the client’s perspective, CacheLayer is indistinguishable from the OpenAI endpoint.

The request flow proceeds as follows. On arrival, the request is authenticated against the users table using the bearer API key. The last user message is extracted and embedded. The embedding is compared against all cached entries for that user via dot product. If a match above the threshold

is found, the cached response is returned. Otherwise, the request is forwarded to the appropriate upstream provider, the response is cached, and it is returned to the client.

The proxy is stateless at the request level. All persistent state lives in a SQLite database that stores cached query-response pairs alongside their embeddings. This means the proxy can be restarted without losing cached data.

3.2 Multi-Tenant Architecture

CacheLayer supports multi-tenant deployments in which multiple independent users or organizations share the same proxy infrastructure while maintaining complete cache isolation. Each user registers via the POST `/register` endpoint, which generates a cryptographically random API key with the `cl_` prefix using Python’s `secrets.token_urlsafe` function. This key is stored in a `users` table alongside the user’s email and creation timestamp.

All cache entries carry a `user_id` foreign key. Every cache lookup filters by `user_id` before computing similarity scores, ensuring that one tenant’s cached responses are never returned to another tenant. This is a hard requirement for production deployments handling proprietary or sensitive query content. Upstream provider keys are passed separately via the `X-Upstream-Key` header, keeping them distinct from the CacheLayer authentication token.

3.3 Embedding Layer

When a request arrives, CacheLayer extracts the last user message from the `messages` array as the cache key. This message is passed to the embedding layer, which loads all-MiniLM-L6-v2 via the Sentence Transformers library and produces a 384-dimensional vector representation. The model is loaded once at server startup and kept in memory, avoiding per-request model loading overhead.

3.4 Cache Store

CacheLayer uses SQLite as its backing store. Each cache entry contains the original query text, the response text, the embedding stored as a JSON array, a Unix timestamp for TTL enforcement, a `user_id` for tenant isolation, and a hit counter for analytics. A separate `stats` table maintains running totals per user for total queries, cache hits, and tokens saved.

SQLite was chosen over an in-memory store because it provides persistence across server restarts at essentially zero operational overhead. For deployments at larger scale, the cache store could be swapped for a vector database such as FAISS [5] or pgvector, which would support approximate nearest neighbor search with better asymptotic complexity. The current implementation performs a linear scan over all cached entries for each incoming query, which is acceptable for caches up to a few thousand entries but degrades at scale.

3.5 Provider Routing

CacheLayer supports both OpenAI and Anthropic as upstream providers. On a cache miss, the `model` field of the request is inspected. If the model name contains the string “claude”, the request is translated to Anthropic’s `/v1/messages` format and forwarded with the appropriate headers. Otherwise, the request is forwarded to OpenAI’s `/v1/chat/completions` endpoint. All responses are normalized to the OpenAI chat completion format before being returned to the client, ensuring provider transparency.

3.6 Threshold Parameter

The similarity threshold is the single most important configuration parameter in CacheLayer. It is a float between 0 and 1 that determines how similar an incoming query must be to a cached query for the cached response to be returned.

We formalize the threshold selection problem as follows. Let Q be the set of all possible queries and let C be the set of cached queries. For an incoming query q , let q^* be the most similar cached query. The cache returns a response if and only if $\text{sim}(q, q^*) \geq \text{threshold}$. Two failure modes exist. A false positive occurs when $\text{sim}(q, q^*) \geq \text{threshold}$ but q and q^* have different correct answers. A false negative occurs when $\text{sim}(q, q^*) < \text{threshold}$ but q and q^* have the same correct answer. Threshold selection is a tradeoff between these two failure modes.

4 Experiments

4.1 Dataset

We constructed a benchmark dataset of 15 query pairs, each consisting of two semantically equivalent phrasings of the same question. The dataset spans three domains: enterprise software and consulting, personal finance, and software engineering. Each pair was written manually to reflect realistic variation in how different users phrase the same underlying information need. The full dataset is provided in the repository.

All query pairs were verified to have semantically equivalent correct answers, meaning a response appropriate for the first phrasing is also appropriate for the second. Pairs where the correct answer might differ depending on interpretation were excluded.

While 15 pairs is sufficient to characterize the threshold-hit rate relationship for this analysis, we acknowledge that this dataset is limited in scale relative to the diversity of real production query distributions. Practitioners deploying CacheLayer in specific domains should validate the threshold recommendations against representative samples of their own query distributions using the benchmark script provided in the repository.

4.2 Evaluation Protocol

For each threshold value tested, we seeded the cache with the first phrasing of each query pair and then queried the cache with the second phrasing. A cache hit was recorded if the cache returned a response. A cache miss was recorded if no response was returned. This protocol isolates the behavior of the semantic matching component from the quality of the underlying LLM responses.

We evaluated six threshold values: 0.70, 0.75, 0.80, 0.85, 0.90, and 0.95. For each threshold we recorded hit count, miss count, hit rate, tokens saved, and estimated cost saved using the GPT-4o output token rate of \$0.000005 per token and an assumed average response length of 60 tokens per query.

4.3 Latency Benchmark

In addition to the threshold analysis, we measured the end-to-end latency of cache hits and misses. Cache hits were measured as the time from request receipt to response delivery including embedding inference and SQLite lookup. Cache misses were measured similarly, excluding upstream API latency. The upstream API baseline of approximately 1,800ms reflects typical GPT-4o response times under normal load.

4.4 Results

Table 1 presents the threshold analysis results.

Threshold	Hit Rate	Hits	Misses	Tokens Saved	Cost Saved
0.70	100.0%	15	0	900	\$0.0045
0.75	93.3%	14	1	840	\$0.0042
0.80 (optimal)	93.3%	14	1	840	\$0.0042
0.85	66.7%	10	5	600	\$0.0030
0.90	33.3%	5	10	300	\$0.0015
0.95	0.0%	0	15	0	\$0.0000

Table 1: Threshold vs. hit rate analysis across 15 paraphrased query pairs.

Table 2 presents the latency benchmark results.

Metric	Value
Average cache hit latency	8.3ms
Average cache miss latency	8.5ms
Live API baseline (GPT-4o)	~1,800ms
Speedup on cache hit	216×
Token cost on cache hit	\$0.00

Table 2: Latency benchmark results. Miss latency excludes upstream API time.

4.5 Analysis

The results reveal a sharp discontinuity in hit rate between threshold values of 0.80 and 0.85. Hit rate drops from 93.3% to 66.7% as the threshold increases by 0.05, then continues declining steeply through 0.90 and collapses entirely at 0.95. This pattern suggests that the cosine similarity scores for paraphrased query pairs using all-MiniLM-L6-v2 cluster in the range of 0.80 to 0.84 for this dataset. Queries that score above 0.85 tend to be near-identical phrasings rather than genuine paraphrases, while queries that score below 0.80 tend to share topical similarity but differ enough in framing to potentially warrant different responses.

The threshold of 0.70 achieves a perfect hit rate on this dataset but likely produces false positives in practice. Queries that are topically related but semantically distinct can score above 0.70, and returning cached responses for these would produce incorrect answers. We therefore do not recommend 0.70 for production use despite its perfect recall on this benchmark.

The threshold of 0.80 achieves the same hit rate as 0.75 while providing greater semantic precision. We recommend 0.80 as the default for general-purpose query caching with this embedding model.

4.6 Cost Projection

Table 3 projects cost savings for three deployment scenarios using the 0.80 threshold, a 93.3% hit rate on paraphrased queries, an assumed paraphrase rate of 60%, and a GPT-4o output token rate of \$0.000005 per token with an average response length of 200 tokens.

Deployment Scale	Daily Queries	Daily Savings	Annual Savings
Small (internal tool)	1,000	\$0.56	\$204
Medium (SaaS product)	10,000	\$5.60	\$2,044
Large (enterprise)	100,000	\$56.00	\$20,440

Table 3: Projected cost savings by deployment scale at threshold 0.80.

5 Discussion

5.1 Threshold Sensitivity and Embedding Model Dependence

The sharp transition we observed between 0.80 and 0.85 is a property of the all-MiniLM-L6-v2 model’s embedding geometry on this particular dataset and should not be generalized to other embedding models without validation. Different models produce different similarity score distributions for the same query pairs. A larger model such as all-mpnet-base-v2 would likely produce different cluster boundaries, and a domain-specific fine-tuned model would potentially produce tighter clusters that allow for a higher threshold with equivalent recall.

Practitioners deploying CacheLayer in a specific domain should calibrate the threshold against a representative sample of their own query distribution rather than relying on the default. The benchmark script included in the repository is designed to support this calibration process.

5.2 Limitations

Several limitations of this work should be noted. The evaluation dataset contains 15 query pairs, which is sufficient to characterize the threshold-hit rate relationship but small relative to the diversity of real production query distributions. A larger benchmark spanning more domains, more query types, and more embedding models would provide stronger empirical grounding for the threshold recommendations we make.

The current implementation performs a linear scan over cached entries for each incoming query. This is computationally acceptable for small caches but does not scale to production deployments with hundreds of thousands of cached entries. Integrating an approximate nearest neighbor index such as FAISS [5] would resolve this limitation without changing the external behavior of the system.

We did not evaluate adversarial queries—meaning queries designed to be similar enough to a cached query to trigger a cache hit while having a genuinely different correct answer. Understanding the false positive rate under adversarial conditions is important for security-sensitive deployments and remains an open direction for future work.

Finally, our cost projections assume a fixed paraphrase rate (60%) and response length (200 tokens per query). Real applications vary significantly on both dimensions, and organizations considering CacheLayer should measure their own query distributions before making deployment decisions.

5.3 Future Work

Several directions would strengthen and extend this work. First, evaluating additional embedding models and characterizing how the optimal threshold varies across models would make the threshold selection guidance more broadly applicable. Second, integrating FAISS [5] or a dedicated vector database would enable evaluation at the scale of millions of cached entries. Third, extending the

proxy to support streaming responses would make CacheLayer compatible with a broader range of application architectures. Fourth, developing a method for automatic threshold calibration based on observed query distributions would reduce the configuration burden on developers adopting the system. Fifth, implementing domain-specific threshold profiles would allow different similarity cutoffs for different query categories within the same deployment.

6 Conclusion

The cost of LLM API usage scales with query volume rather than query diversity. When users ask semantically identical questions with different surface forms, exact match caching fails to eliminate the redundant API calls. CacheLayer addresses this by operating as a network-level proxy that uses sentence embeddings and cosine similarity to identify semantically equivalent queries and serve cached responses without contacting the upstream API.

Our empirical evaluation shows that a cosine similarity threshold of 0.80 achieves a 93.3% cache hit rate on paraphrased query pairs using the all-MiniLM-L6-v2 embedding model, with hit rate declining sharply above this value. Cache hits return in 8.3ms, a $216\times$ reduction relative to live API calls. At production scale, this translates to meaningful cost reductions across deployment sizes, from a few hundred dollars annually for small internal tools to tens of thousands of dollars for high-traffic enterprise applications.

The proxy architecture, multi-tenant isolation, and provider-agnostic routing are the key practical contributions of this work. By implementing the OpenAI chat completions specification and supporting both OpenAI and Anthropic providers, CacheLayer requires no changes to existing application code. Developers can adopt it by changing a single URL, which removes the integration friction that has historically limited adoption of caching solutions requiring library-level changes.

Code, benchmark data, and evaluation scripts are available at <https://github.com/ZainRizvi9/CacheLayer>. A live deployment is accessible at <https://cachelayer.up.railway.app>.

References

- [1] Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems*, 33, 5776–5788.
- [2] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics.
- [3] Zilliz. (2023). GPTCache: A library for creating semantic cache for LLM queries. *GitHub*. <https://github.com/zilliztech/GPTCache>
- [4] BerriAI. (2023). LiteLLM: Call all LLM APIs using the OpenAI format. *GitHub*. <https://github.com/BerriAI/litellm>
- [5] Johnson, J., Douze, M., & Jegou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- [6] OpenAI. (2024). OpenAI API reference. <https://platform.openai.com/docs/api-reference>

[7] Anthropic. (2024). Anthropic API reference. <https://docs.anthropic.com/en/api>